LIVING CHECKLIST · UPDATED WITH EVERY CHANGE

FOX_Checklist

What this is: every FoxByte feature and its sub-features — built or not, and how it was tested — so it's clear where we are and what's next. How each feature works is in `FOX_Feature_Implemented.pdf` (same ids).

Last updated: 22 September 2026 · **Reflects:** `main` @ `2778762` (v1.0.1, exact repository path PR #1) + the FoxByte rename and brand indirection (`chore/foxbyte`)

Source: `docs/FOX_Checklist.html` → `make feature-doc` → this PDF

✅ Done & tested — automated tests (unit, VM integration or CI) ☑ Done — tested by hand only ◐ Partial — gaps, or not verified end to end ☒ Known issue ☐ Not started

Sub-features: ✓ built ◐ partly ! a known limit → planned, not built □ not built yet ✕ broken **The badge rule** (audit v2 G31, which found 47 rows badged done while carrying an open item): a row is *done* or *done (manual)* only when nothing in it is ◐, □ or ✕, and `scripts/test_feature_checklist.sh` fails the build otherwise. A **!** is a limit of what the feature does and a → is a planned extension — neither makes the row's own claim untrue. How thoroughly something was tested belongs in "Tested by", not in the sub-features. · "VM §n" = section of `make integration` ; "v2 §n" = `make integration-v2` ; "update" = `make integration-update`

Where we are

Area	Features	Done & tested	Done (manual)	Partial	Known issue	Not started	Progress
A · Install & setup	9	4	3	2	·	·	78%
B · CLI & services	3	3	·	·	·	·	100%
C · Branching & storage	8	7	·	1	·	·	88%
D · Durability	6	5	·	1	·	·	83%
E · Security foundations	5	5	·	·	·	·	100%
F · SQL gateway	7	6	1	·	·	·	100%
G · Accounts & auth	6	5	·	1	·	·	83%
H · Blackbox (record of schema changes)	22	18	·	4	·	·	82%
I · Import & migration	13	5	8	·	·	·	100%
J · ETL pipelines (experimental)	4	2	2	·	·	·	100%
K · APIs, SDKs & agents	6	4	2	·	·	·	100%
L · Web console	11	1	10	·	·	·	100%
M · Build, CI & releases	9	7	1	1	·	·	89%
N · Realtime change feed	1	1	·	·	·	·	100%
O · Personal data & compliance	3	3	·	·	·	·	100%
Total (features built or started)	**113**	**76**	**27**	**10**	**·**	**·**	**91%**
Z · Not started (roadmap)	7	1	·	·	·	6	14%

Storage engine — not implemented, by design

FoxByte does **not** have its own storage engine. Data is stored by **PostgreSQL's own engine** (heap tables, B-tree indexes, write-ahead log) running on local disk inside the stock `postgres:18-bookworm` image plus wal-g (`postgres:16-bookworm` for an install created before 21 Sep 2026). This was a locked architecture decision: a Neon-style engine that separates compute from storage over the network adds latency to every transaction, and transaction speed must never be compromised.

What **is** built is a storage *layer* around Postgres: copy-on-write branch drivers — ZFS on macOS/Linux, btrfs on Windows — behind one interface in `internal/branch/storage.go` (C1), quotas (C6), capacity reporting (C7), and durability to object storage with wal-g + MinIO: WAL archiving, base backups, point-in-time restore (D1–D3). Networked storage disaggregation is listed in the README roadmap but not started (see "Not started").

Suggested next steps

1. **The claims are checkable now**: a restore is proved weekly and by `fox backup verify` (D2), a tag cannot ship past a red or stale nightly (M2), every command the engine runs has a deadline so nothing hangs for ever (G30's useful half), containers restart on crash (B2), and CI runs the race detector. Before this, the most consequential thing the product claims — that the backups restore — had never been tested end to end on a real install.
2. **This checklist's own badge rule is enforced** (audit v2 G31): no row can be badged done while carrying an open sub-item, and `scripts/test_feature_checklist.sh` fails the build if one is. The 45 sub-items that broke the rule were each put somewhere honest — built, a limit (!), planned (→), or a note in "Tested by" — and nine of them turned out to be things we had already shipped and never ticked off, including automated UI tests, per-user access control, SDK contract tests and the first signed release.
3. Run on real machines what the throwaway VM cannot: `fox update` across a real release on macOS and Windows (A9); on Windows, the zstd distro image's unpack (M5), the btrfs holding area of `fox pg upgrade` (D6), and an export's copy to the host (D5).
4. Automated tests for the imports (I1–I3, I7) — the last area with none.
5. Gate release tags on the nightly suites, so a tag cannot ship past a red suite (M2).
6. Decide how this checklist shows partial work (audit v2 G31): 9 features are badged done while carrying an open sub-item (`C7, D5, D6, F5, G3, H16, H17, I13, M7`) — down from 34. Each of those sub-items is a disclosed limit rather than unfinished work, so the remaining question is whether a limit belongs in the sub-item list at all.
7. **The object store's images are FoxByte's own** (M6): MinIO left Docker Hub in 2026 and then quay.io stopped serving it anonymously — every tag there answers 401 — so a fresh install on macOS or Linux could not start, while existing installs carried on from their cache and Windows carried on because its distro image ships them inside it. They are mirrored to `ghcr.io/thefoxbyte` by `.github/workflows/mirror-images.yml`, which takes them out of a published release's distro asset rather than from a registry

that will not serve us. **Run it once and pin what it prints.** It can only be dispatched after it is on the default branch, and until the digests are pinned `release.yml` refuses to publish.

8. **Promotion is built** (H18, audit G29): a branch's recorded statements, reviewed, then applied to another branch in one transaction — attributed to whoever wrote them, with the approver beside them. Schema only. An agent can propose and cannot approve its own request.
9. **From the audit, deliberately left:** HA beyond the demo — automatic failover and fencing across hosts (D4, G21); metrics and alerts, and supervision of the background servers themselves (B2, G27 — the containers restart); splitting the branch package and running its SQL through pgx (G30 — the deadlines are in); quotas on btrfs and per-branch CPU/memory (C6, G33).

A Install & setup

7 of 9 done

	ID	Feature · sub-features		Tested by
☑	A1	**One-line installer (macOS / Linux)** ✓ Detects OS / CPU, downloads the release binary ✓ Installs to the prefix (sudo only if needed) ✓ Stages the Linux engine for the VM on macOS ✓ `FOX_VERSION`, `FOX_REPO`, `FOX_DIST`, `FOX_PREFIX` ✓ Verifies every download against `SHA256SUMS`; refuses what it can't check	✓ Release signature checked (OpenSSL 3; warns with LibreSSL) ✓ One-liner names `get.thefoxbyte.com`, from `brand.json` (`deploy/get`) → The `get` site goes live: Vercel project + DNS record (owner's step)	Unit (`test_install_sh.sh`, `TestInstallHostServesTheInstallers`, CI); manual
◐	A2	**Windows installer (install.ps1)** ✓ Installs WSL, elevates, resumes after reboot ✓ Downloads fox.exe, engine, image build context ✓ Ubuntu rootfs verified against Ubuntu's checksums ✓ Adds to PATH and runs `fox setup`	✓ Prefers the prebuilt distro image, falls back to the rootfs ✓ Verifies FoxByte's own downloads ✓ Run on a real Windows machine (distro image downloaded and verified)	Pester (Windows CI); Windows run 16 Sep
☑	A3	**`fox setup` on macOS (Lima)** ✓ Creates / starts the Lima VM ✓ Installs Docker and ZFS ✓ Reinstalls the engine on every run ✓ Starts the stack	✓ Repairs an interrupted provisioning on the next run (dpkg interrupted, half-installed packages) ✓ Image builds from the copy inside fox when it cannot be pulled (C8)	Unit; run against a deliberately broken VM, 21 Sep
☑	A4	**`fox setup` on Windows (WSL2 + btrfs)** ✓ Imports a dedicated distro with systemd ✓ Docker, btrfs and a storage mount unit ✓ Quiet steps, detail in install.log ✓ Waits for systemd on a cold distro	✓ Preloaded images tagged as the engine runs them ✓ Summary with API key and `sslmode` ✓ Fresh install from v0.8.6 on Windows: preload used, no apt, key in the summary	Unit (`host_wsl_test.go`); Windows run 16 Sep
☑	A5	**Linux native** ✓ Engine runs in-process, no VM ✓ `fox setup` explains nothing is needed	✓ Services listen on loopback by default (`FOX_LISTEN` to expose)	VM integration (all suites run on Linux)
☑	A6	**Command forwarding & host file streaming** ✓ macOS → Lima VM, starts the VM if stopped ✓ Windows → WSL distro	✓ `fox import` streams a host file over stdin → Stream `pipeline run` spec files too	Unit (`TestImportLocalFile`, `TestWSLArgs`)
☑	A7	**Always-latest engine binary** ✓ Downloads the newest engine into the cache (ELF check) ✓ `FOX_NO_REFRESH`, `FOX_VERSION`	✓ Verified against `SHA256SUMS`; keeps the staged engine if not ✓ Seen verifying on a real Windows install	Unit (`host_test.go`, `update_test.go`); Windows run 16 Sep
☑	A8	**New-release notice on `fox start`** ✓ Background check, 1.5 s limit, printed last ✓ Only a promoted, newer, complete release	✓ One request per start; a found release remembered 6 h, never "up to date" ✓ Off switch; never on dev builds; silent offline	Unit; update §1, §1b, §2, §3, §8; real Mac check
◐	A9	**`fox update`** ✓ Downloads and verifies everything before changing anything ✓ Stops servers, installs engine, restarts old one on failure ✓ `up`, Blackbox upgrade, `start` on the new engine ✓ Replaces host `fox`, keeps previous binaries ✓ `--check`, `--yes`, `--version` ✓ Linux path verified end to end ✓ macOS path run against real releases (v0.8.3 → v0.8.4 → v0.8.5)	□ Windows path run against a real release (deferred) ✓ Release signatures (Ed25519 over SHA256SUMS) ✓ The version line stays readable by older installs, so `fox update` works from any released version ✓ A previously released version is shown to accept each build (nightly `upgrade` suite) and each release (`release-verify`)	Unit; update §4–§7; a real Mac run

B CLI & services

3 of 3 done

	ID	Feature · sub-features		Tested by
☑	B1	**The `fox` command set** ✓ Stack: setup, start, stop, up, down (same as stop), status, check, logs, connect, psql, demo, update ✓ Branching, backup / restore, HA ✓ Blackbox, policy, impact ✓ Import, pipelines ✓ backup export / restore, pg status / upgrade	✓ Users, API keys, admin ✓ `fox vm status` / `fox vm shell` (Lima, WSL; Linux says there is no VM) ✓ `branch create --from` ✓ `fox uninstall` removes it all, retired installs included	Unit (`main_test.go`, `update_test.go`); VM suites
☑	B2	**Background services (daemon)** ✓ Control plane :8080, gateway :6432, agent API :8088 ✓ Detached, pidfiles and logs in `~/.fox` ✓ Stop (TERM then KILL), status, logs	✓ Containers restart on crash and after a reboot (`--restart=unless-stopped`); a container stopped on purpose stays stopped ! The three background servers themselves are not supervised (`fox check` reports them)	Unit (`TestStopTerminatesProcess`); VM §1

	ID	Feature · sub-features		Tested by
✅	B3	**First run (sample data, `check`, `connect`)** ✓ Sample data seeded into `main` on a first start: `users`, `projects`, `events`, five rows each, two foreign keys ✓ `fox demo seed|drop|sql` on any branch; `FOX_NO_DEMO=1` starts empty ✓ The console opens itself, with the one-time setup token already in the link ✓ Declines to open a browser in the VM, over SSH, with no display, or with `FOX_NO_BROWSER` ✓ `fox check`: version, console, storage, main, three servers, three ports, TLS, Blackbox, backup age, sample data — each failure with the command that fixes it ✓ `fox check` counts unused anonymous Docker volumes left by older versions, naming the prune that keeps named volumes ✓ **It survives the install it is run on** (13 Oct 2026, reported from a real machine): with no storage pool and no containers, `check` used to run for over ten minutes and print nothing at all, leaving a `postgres-walg` container behind it. Lines are now printed as they resolve, each check has a deadline of its own (`FOX_CHECK_TIMEOUT`, 10s) and says it could not be determined rather than blocking, the base-backup listing — the one shell-out the deadline tiers missed — is bounded and removes the container it started, and a listing that cannot be made is no longer reported as "no base backup yet". Measured: 0.2s with the pool destroyed, 5.3s against an unreachable bucket, byte-identical output on a healthy install ✓ **A failing command keeps what it said** (13 Oct 2026, the other half of the same report): `capture`, the engine's way of asking a container a question at 42 call sites, dropped the stderr `cmd.Output()` had already collected, so `check`'s Blackbox line could only say "verify query: exit status 1" while psql had written the reason to the other stream. The first line of stderr now reaches the error — one line, bounded at 300 runes and cut by rune — for every caller at once, while parsing stays on stdout so a `NOTICE` cannot corrupt output that callers split on separators ✓ `fox connect [branch]` prints the application's connection string, then opens psql; `fox psql` takes a branch ✓ `fox branch create` ends with the connection string and how long it took ✓ Plain-English errors in the CLI and the console (`internal/errhint`, `web/src/errhint.ts`) ✓ `fox check` reports the storage pool's health (a suspended or faulted pool fails, with the way out) and the free space on the disk under the pool's file and Docker's data		Unit; v1 §11d

C Branching & storage

7 of 8 done

	ID	Feature · sub-features		Tested by
✅	C1	**Storage drivers (ZFS / btrfs)** ✓ ZFS snapshot + clone per branch (macOS VM, Linux) ✓ btrfs subvolume snapshots (Windows) ✓ Pool / filesystem created on a loopback file	✓ One storage interface for both ✓ Delete removes the clone's own origin snapshot, so a non-main parent can be re-used	Unit; VM §2, §2b (ZFS); btrfs manual on Windows
✅	C2	**Branch lifecycle** ✓ Create, list, delete ✓ Reset to parent ✓ Suspend / resume ✓ Recreate a container from its surviving data ✓ Create from any branch (`--from`, REST `from`, Dashboard picker)	✓ Names validated in the CLI and at the API boundary (one check, branch.checkName) ✓ A removed container takes its anonymous Docker volume with it (`docker rm -f -v`); named volumes are kept	Unit; VM §2, §2b, §4, §11e2
✅	C3	**Protection of `main`** ✓ Delete and reset refused	✓ Suspend refused for main, the HA standby and whichever container serves main	Unit; VM §4, §6
✅	C4	**Private networking (no host ports)** ✓ Branch databases reachable only through the gateway ✓ Published ports (object store, restore, debug) on loopback only; standby unpublished	✓ An older install's public object store is re-created on start	Unit (`TestParsePublishedPort`, `TestDSN`); VM §1c
✅	C5	**Scale-to-zero & wake on connect** ✓ Idle branches suspended automatically ✓ Next connection wakes the branch (~0.3–0.7 s)	✓ HA standby never suspended → Idle clocks survive a gateway restart	VM §7
◐	C6	**Storage quotas** ✓ Space reserved for main (ZFS) ◐ Per-branch cap (opt-in)	□ Quotas on btrfs	Manual
✅	C7	**Capacity & status** ✓ Services, main health, backups, branches ✓ Storage used / available	! btrfs "used" is apparent size	VM §1
✅	C8	**Provisioning & image pull** ✓ Docker, storage pool and image prepared on `up` ✓ Pulls from GHCR, builds locally as fallback	✓ The build context is built into fox, so the fallback works on any install	Unit; VM §11b

D Durability

5 of 6 done

	ID	Feature · sub-features		Tested by
✅	D1	**WAL archiving to object storage** ✓ wal-g ships main's WAL to MinIO, off the commit path ✓ MinIO images pinned on quay.io ✓ Branches are disposable by design (export one that matters) ✓ Remote S3 target (`fox backup target set`), keys in 0600 env files	✓ Object Lock reported; tested on MinIO ✓ `fox check` warns while the archive is still on the same disk as main, naming the consequence and the one command that fixes it (HA plan, phase 0)	VM §3; VM §11c; not yet run against AWS S3

☑	D2	**Base backups**		Unit; VM §3, §3b, §6; VM §11c, §11e
		✓ Create and list ✓ Follow the primary after a failover; listing works with main stopped ✓ Read-only listing: `GET /api/backups` and a Dashboard panel ✓ Schedule and retention (`FOX_BACKUP_INTERVAL` , `FOX_BACKUP_RETAIN`) ✓ A first backup at install ✓ Age shown and warned in `fox status` and the status API	✓ Proven, not assumed: `fox backup verify` writes a probe row, waits for its WAL segment to be archived, restores the newest base backup into a container of its own and reads the row back ✓ On a schedule (weekly; `FOX_RESTORE_CHECK_INTERVAL`), with the result in `fox check` ✓ A check that main's restart or absence stopped is not recorded as a failed restore; the scheduler retries in 5 minutes	
☑	D3	**Point-in-time restore**		Unit; VM §3, §3b, §6; v2 §1a
		✓ Restore to a timestamp or latest on :5433 ✓ Per-install credentials	✓ Starts from the newest base backup that precedes the target ✓ A point no backup precedes is refused, naming the oldest	

D4 HA standby & failover

Unit; VM §6, §7

- ✓ Streaming hot standby
- ✓ Manual failover, `main` rerouted
- ✓ Nothing deletes or stops the promoted standby
- ✓ `fox up` brings it back, not the old main
- ✓ `fox ha failback`, keeping every write
- ✓ The promoted standby archives WAL, so backups and PITR cover the failover
- ✓ With the change feed on, the standby's own physical slot and a conninfo with a database name, so realtime subscribers keep their position across a promotion (N1)
- ✓ **A replication role of its own** (HA plan phase 1, first item): `db_replica` copies the database and can read none of it, admitted by one `pg_hba` rule naming it and this install's own network. Replaces `host replication all all` plus a standby connecting as the bootstrap superuser — the wide rule is deleted rather than outranked, the superuser can no longer replicate over the network while its ordinary connections are untouched, and `fox check` warns on an install whose HA was enabled by an older version and still carries the old rule
- ✓ **TLS on replication, and publishing a primary at all** (HA plan phase 1): `fox node expose` — opt-in, one address, TLS required with a per-install CA the standby verifies, `db_replica` only, one peer `/32` only. Verified between two machines: without TLS refused, the superuser refused even over TLS, a real base backup over TLS succeeded. The private key is kept out of `PGDATA`, which `pg_basebackup` copies. The join token carries the CA and one role's password — never the install secret, from which the superuser's could be derived
- ✓ **A standby node is told what its licence costs it** (HA plan §3.1): `node join` warns when this machine has no licence that unlocks the paid features — a licence is per-install state and joining copies the database, not `~/.fox`, so a fresh standby has none and refuses paid features immediately on promotion, with no grace. Corrects the plan, which assumed a mismatched fingerprint lapses after 14 days: it never lapses, so the dangerous state is an absent licence, not a mismatched one
- ✓ **Reads from the standby** (HA plan phase 7): a read-only session asks for the copy by name — `main:standby`, same key, same gateway — and the standby answers it instead of the primary, so a long report stops competing with the writes. On one machine that is process isolation rather than read scaling, and across two machines it is refused with the reason: a joined node's copy carries the primary's roles while that machine's gateway has its own secret, which is plan §3.2's shared state. The suffix is resolved before the ACL, the key's scope and the audit record, so it chooses a backend and never widens what a key may open; writes there are refused by Postgres itself; and the name is refused — naming the command that fixes it — when there is no standby, when the branch is not main, when the standby is not running, and above all when the standby has been promoted, where quietly serving the primary instead would undo the whole point. Staleness is reported rather than capped: `ha status` says how far behind the copy is and `connect` prints the endpoint with the caveat
- ✓ **Accounts and keys reach every node** (HA plan phase 2): the primary writes `users`, `api_keys`, `oauth_identities`, `branch_owners`, `pipelines`, `pipeline_targets` and `change_requests` into `statedb` in the replicated cluster, and a machine that follows a primary elsewhere applies them into its own SQLite — so a failover no longer leaves nobody able to sign in. The store deliberately does *not* move into `main`: `internal/auth` never imports the engine, so signing in works while the database is down, which is the window the phase exists for. Exactly one node writes (a node ships only while it follows nobody, and a promotion clears that record), the document travels in the WAL already being streamed, a revoked key is deleted on the other node rather than merely not updated, `statedb` is closed to `db_client` and every role clients use, and an account change on a follower is refused instead of being silently overwritten. Sessions, the two append-only logs and the hash-chained security log are deliberately left behind, each for a stated reason
- ✓ **A promoted copy rewrites its role passwords** (HA plan phase 2): a standby from another machine carries that install's roles, whose passwords derive from its secret — which `node join` never carries — so a promotion used to leave every client unable to authenticate. The promoted node now rewrites the passwords of the roles this install manages, over the container socket that needs no password, leaving anything else alone; `fox ha repair-roles` is the hand crank. Clients are unaffected: they authenticate at the gateway with an API key. Measured: break one role's password and the gateway refuses the connection; repair it and the same key works again
- ✓ **Automatic failover, refused without a witness** (HA plan phase 3): `fox ha watch` on a standby promotes only when it and a witness on a third host both fail to reach the primary for `--after`. Two machines cannot tell a dead primary from an unreachable one, so the command refuses to run without a witness rather than working most of the time. Four decisions, one of which promotes; "the witness says the primary is fine" and "the witness could not be asked" are refused separately and logged differently. The promotion is the ordinary fenced failover — `--force` is not reachable from the watcher — and both promotions and refusals go into the hash-chained security log, which phase 2 keeps on each node. `fox ha witness` answers one question over HTTP and tells a stranger nothing: an unauthenticated ask gets 404, not 401. Not pg_auto_failover or Patroni: both want to own the data directory, the recovery configuration and the promotion, which this engine owns. What is left is two gateways behind a virtual IP, which waits on whether the customer supplies it
- ✓ **A drill that says what it actually tested** (HA plan phase 6): a result carries the topology it ran in and who noticed, so the report stops disclaiming a second site it had or manual detection it did not use — with two machines and a watcher it says "Nothing it set out to" and names the one thing no drill can test, which is whether the witness is where the clients are. Detection is read from the security log, not passed as a flag, because the watcher is another process. `fox ha drill --takeover` measures a site failover from the surviving machine: it stops nothing and promotes nothing, RTO therefore includes noticing (the number a client experienced), and RPO is reported as not measurable from that side rather than as a zero that would read as a guarantee. A local drill is refused on a machine that follows a primary elsewhere, naming the one that applies
- ✓ …and `fox check`'s licence line says it on every run, not just at join time — naming what a promotion would cost, or that a licence activated elsewhere survives one. The clause adds the consequence and never changes the severity
- ☐ The licence *policy* for HA pairs is still a decision (plan §6.1) — site licence, fingerprint set, or per-node. Recommended: site licences, the only option needing no new code
- ✓ **Fencing across machines** (HA plan phase 3, first half): `ha failover` on a machine following a remote primary refuses while that primary answers, because promoting would make a second one — a split brain that phase 1 shipped and that was measured (both machines accepting writes). Verified refused with the primary alive, promoted once it was stopped; `--force` warns before proceeding; a promotion says the old primary must re-join rather than restart as a primary. It cannot tell a dead primary from an unreachable one, which needs a failover manager
- ☐ The rest of phase 3 — a failover manager for automatic detection — is a decision first: `pg_auto_failover` is installable for PG 18 but its last release predates it, and either manager wants to own `synchronous_standby_names`, which phase 5 made ours

- ✓ **A standby on another machine** (HA plan phase 1, complete): `fox node join <token>` copies the primary over TLS and runs it as a standby, verifying that primary's certificate. Demonstrated by killing the primary's database with SIGKILL and promoting the survivor, which took writes with every acknowledged row intact. A promotion clears the join, since the survivor follows nobody; `ha enable` refuses on a joined machine rather than silently rebuilding the standby from the local main. Two VMs share a kernel and a disk, so this proves networking, credentials, TLS and promotion — not survival of a power cut
- ✓ **Failover drills** (HA plan phase 6): `fox ha drill --yes` kills the primary with writes in flight, recovers through the real failover path, and writes a dated RPO/RTO report to `~/.fox/drills/`. Measured 0 of 60 acknowledged writes lost and 400ms to service. The report refuses to let a figure be misquoted — zero under asynchronous replication is an observation, zero under zero-data-loss mode is a guarantee — and names what it did not test: losing a machine, noticing, and a second site
- ✓ **Zero data loss, opt in** (HA plan phase 5): `fox ha sync on|off` — a commit waits until the standby has it on disk. Off by default; with one standby, if it stops writes stop, and the CLI says so. The release works while commits are stalled, a failover clears it so the promoted node is never left waiting for a standby it does not have, and `fox check` calls "on but nothing counted" a failure
- □ Automatic failover, fencing, multi-host

	ID	Feature · sub-features		Tested by
✓	D5	**Portable export & restore** ✓ Every non-agent branch: pg_dump, its roles, the Blackbox head; SHA256SUMS over all ✓ Copied out of the VM and verified on the host; a copy that fails is deleted	✓ Damaged, incomplete or path-escaping archives refused ✓ Restore into a new branch, same or newer major; Blackbox checked exact	Unit; VM pg-upgrade §1; the host copy on macOS/Windows is unit-tested only
✓	D6	**PostgreSQL major upgrade** ✓ `fox pg status`: the major of the install and of each branch ✓ Plan with every change crossed, found-in-your-data first; refusals say what to do ✓ Asks, exports, then reloads every carried branch on the new major	✓ Data, roles, API keys and every Blackbox entry come across ✓ Old databases held outside the branch namespace; rollback and finalize ✓ Each upgraded cluster archives WAL under its own prefix → Carry agent branches; upgrade with HA on	Unit; VM pg-upgrade (72 / 72); the btrfs (Windows) holding area has not been run

E Security foundations

5 of 5 done

	ID	Feature · sub-features		Tested by
✓	E1	**Per-install generated secrets** ✓ Generated Postgres and MinIO passwords ✓ Used by restore too	✓ Each login role has its own derived password ✓ Never on a command line (docker `--env-file`)	Unit (indirect); v2 §1a; VM §1d
✓	E2	**TLS everywhere** ✓ Self-signed TLS on all three services, zero setup ✓ TLS required from other machines (loopback clients may skip it)	✓ No plain-HTTP fallback except on loopback	Unit (`TestReadStartupUpgradesTLS`)
✓	E3	**Keys are made by a person, not by start** ✓ Start creates no account and no key, and caches none on disk ✓ The banner shows the connection string and how to make a key	✓ The first account created gets the admin grant on main ✓ First account over HTTP needs the setup token (`fox setup-token`) ✓ Exactly one first account, however many sign up at once	Integration §1
✓	E4	**Browser security headers** ✓ No response can be framed; none is type-sniffed ✓ The console runs scripts from its own server only ✓ The console loads its fonts from its own server, no other site	✓ HSTS on TLS to a real name, never to localhost ✓ No console page breaks the policy (UI suite)	Unit; VM §1c; UI
✓	E5	**Cross-site requests refused** ✓ A change from another site's page is refused on both APIs and at sign-in ✓ Origin: null and Sec-Fetch-Site: cross-site refused	✓ The console's own origin and non-browser clients unaffected	Unit; VM §1c

F SQL gateway

7 of 7 done

	ID	Feature · sub-features		Tested by
✓	F1	**Routing by database name** ✓ One endpoint :6432, database name = branch	→ Connection pooling, rate limiting	Unit; VM §1, §6
✓	F2	**API-key authentication** ✓ `key_` key as the Postgres password ✓ Bad key rejected with a Postgres error	✓ Startup deadline and connection cap ✓ An account key opens main, its own branches, or any for an admin	VM §1

✅	F3	**Backend login (SCRAM)** ✓ Cleartext, md5, SCRAM-SHA-256	✓ Clients never see the database password	VM §1, §6, §8 (exercised); no unit test for SCRAM yet
☑	F4	**Per-user Postgres login roles** ✓ Role per user, actor can't be forged ✓ Permissions of non-superuser `db_client`	✓ Own password per role	Manual
✅	F5	**Attribution session settings** ✓ Actor kind, branch, session id per connection	! Kind / branch / session client-settable	VM §8
✅	F6	**Ctrl-C (CancelRequest) forwarding** ✓ Cancel a running query through the gateway		Unit
✅	F7	**Errors & insecure build tag** ✓ Real Postgres error responses	✓ No-auth bypass impossible in release builds	Unit (`TestSendError`)

G Accounts & auth

5 of 6 done

	ID	Feature · sub-features		Tested by
✅	G1	**Auth store (SQLite)** ✓ Users, keys, sessions, OAuth ids, pipelines ✓ Safe under concurrent use (WAL, busy timeout), including a new store created by three services at once	→ Roles / permissions, multi-tenant projects	Unit (`TestVerifyKeyConcurrent` , `TestConcurrentFirstOpen`); VM §1
✅	G2	**Email & password users** ✓ Create account, log in ✓ Password change (signs out other sessions); admin reset at the CLI → Self-service reset by email ✓ List and delete users (admins)	✓ Emails validated; at most 63 bytes (a role name) ✓ Forgotten password: `fox user passwd` , named on the login page, README and Guide ! No self-service reset by email — it needs the machine, which suits a local install and not a hosted one	Unit; VM §1; VM §1d
✅	G3	**API keys** ✓ Create (shown once), list, revoke ✓ Bearer / X-API-Key and gateway password ✓ Revoking an id that is not there is refused (404), not reported as revoked	✓ Agent keys (`--agent` , "for an AI agent"): the owner's branches, as an agent; refused by the control plane; never an admin	Unit (`TestAgentKey`); VM §1; v2 §1c
✅	G4	**Web sessions & signup control** ✓ httpOnly session cookie ✓ Sign-up closed by default (`FOX_SIGNUP=open`)	✓ Failed sign-ins throttled	Unit (`TestSessions`)
🟠	G5	**OAuth (GitHub / Google)** ✓ Sign-in flows built ✓ Defaults match what the engine serves (`https://localhost:8080` ; web origin = public URL) ✓ Session cookie `Secure` / `SameSite` follow the origin	☐ Tested with a real OAuth app ✓ Only a provider-verified email joins or creates an account; closed sign-up respected ✓ State cookie one-use, `Secure` , `SameSite=Lax`	Unit; VM §1b
✅	G6	**Security log** ✓ Sign-ins, failures, keys, passwords, accounts, admin grants, refusals ✓ Hash-chained, append-only, anchored and signed	✓ `fox audit` , `verify` , `GET /api/audit` → Included in `fox backup export`	Unit; VM §1d

H Blackbox (record of schema changes)

18 of 22 done

	ID	Feature · sub-features		Tested by
✅	H1	**DDL recording** ✓ Who, tool, branch, statement, status, risk ✓ Inherited by every branch ✓ A restart records nothing: the install script's own statements are not captured ! Role / database changes, some dropped object types ✓ Agents' UPDATE and DELETE recorded; TRUNCATE by anyone ✓ An agent's UPDATE or DELETE with no WHERE is refused, and the attempt recorded — a superuser or db_admin can let one through	✓ An agent's UPDATE or DELETE over a table's row limit (100 by default, `fox guardrail limit`) is refused at the first row over, whatever its WHERE ✓ A budget per agent per table over time, across statements (1,000 rows/hour by default, `fox guardrail budget`) → Approval grants ✓ Triggers on every table, protected from drop/disable/rename and from being made replica-only	VM §8; v2 §9; v2 §10
🟠	H2	**Hash chain, append-only & verify** ✓ Rows can't be edited or deleted normally ✓ `fox ledger verify` ◐ Superuser rewrite caught only via anchors (H10–H12)	✓ `bb.policy` changes recorded (append-only, not chained)	VM §8; v2 §0, §6b
✅	H3	**Attribution** ✓ Humans via their login role ✓ Agents via their own `agent-<id>` role ✓ Kind derived from the role, so neither can pose as the other	✓ The engine's agent sessions (MCP) log in as `db_agent` , and agent keys as `agent:<email>` : agents by login ! An engine agent session's actor name is still injected (signed identity is the expert review plan's F1) ✓ Tool and session are client-declared by design, and labelled so	Unit; VM §8; v2 §1c

✅	H4	**Destructive-DDL guardrail** ✓ DROP TABLE / DROP SCHEMA refused, attempt recorded ✓ Override only superusers / `db_admin`, use recorded	✓ "flag" and "allow" actions; unknown actions refused ✓ No hang when blocked after a write in the same transaction ✓ TRUNCATE blocked by default, recorded	VM §8; v2 §1b, §6b
✅	H5	**Risk flags** ✓ Type change, drop column flagged ✓ Judged on the running statement (no comments or literals)	✓ Automated test	Unit; v2 §6b
✅	H6	**Blackbox access (CLI / REST / MCP / UI / SDK)** ✓ CLI, REST with filters, MCP, web page, SDKs	✓ Per-user access control (internal/access: owner or admin)	v2 §0, §5
✅	H7	**`fox ledger revert --to`** ✓ Help and output say it's a point-in-time restore of main	✓ Points to H13 for a branch from before a change	Unit (help text); VM §3 (the restore)
✅	H8	**Override permission (db_admin) & upgrade** ✓ `fox admin grant|revoke|list` ✓ First user granted automatically ✓ `fox blackbox upgrade [--all]`	✓ Grant / revoke from the web (admins only) ✓ A grant or revoke reaches every server process within a second	v2 §1b, §6; VM §1d
✅	H9	**Blackbox 2.0 capture** ✓ Transaction id and WAL position per entry ✓ Task, tool-call hash, override use	✓ Fail-safe, append-only ✓ Kill switch not settable by clients	Unit; v2 §2
✅	H10	**Checkpoints & anchors** ✓ Merkle root in the database and in read-only anchor files ✓ On a schedule and on demand ✓ Published anchor format ✓ Copied to the backup target (S3 Object Lock) ✓ Anchors signed (Ed25519, key outside the databases)	✓ Anchor target: a bucket of its own, every anchor locked in COMPLIANCE mode (`fox blackbox anchor-target`); `fox check` reports it ✓ fox-verify reads the bucket's object versions; deletes, overwrites and gaps are findings (G3b)	Unit; v2 §3, §3b; VM §11c
✅	H11	**fox-verify (independent verifier)** ✓ Checks a live branch or an export ✓ Release binaries for every platform ✓ `--pubkey` checks signatures	✓ `--anchors-s3` : reads every version in the anchor bucket with its own keys	Unit; v2 §3
✅	H12	**Blackbox integrity & export** ✓ INTACT / TAMPERED via CLI, REST, MCP ✓ JSON-lines export	→ Streaming for very large ledgers	v2 §3
✅	H13	**Branch from just before a Blackbox entry** ✓ Restore + replay up to the change's transaction ✓ By transaction id or timestamp ✓ CLI, REST, MCP, the Integrity panel, and every row of the Blackbox page	✓ main untouched ✓ Documented: README, the Guide's step 6, and the reference	Unit; v2 §4; VM ui
✅	H14	**Blackbox names & aliases** ✓ `fox blackbox`, `/blackbox` routes, MCP and SDK names	✓ Every original name still works	Unit; v2 §5
✅	H15	**Policy gate (warn / block)** ✓ BBX02 warn / BBX01 block with JSON detail ✓ Blocked attempts recorded ✓ Per-rule `db_admin` override	✓ CLI, REST, MCP, web page; custom rules ✓ Patterns match the running statement ✓ TRUNCATE (its own guardrail, H4)	Unit; v2 §6, §6b
✅	H16	**Agent provenance & execute_change** ✓ Session, task, parent session on every agent change (also through the Gateway DSN) ✓ MCP `execute_change` with policy preview	✓ `fox blackbox sessions`, REST ! Provenance is what the agent says (not verified)	Unit; v2 §7
◐	H17	**Impact analysis** ✓ Dependents (views, foreign keys, indexes, depth) ✓ Other branches that have the object ✓ Policy verdict and risk score	✓ CLI, REST, MCP, inside execute_change ◐ Functions, types, schemas not recognised from SQL	Unit; v2 §8
✅	H18	**Blackbox diff, and promoting a branch's changes** ✓ Fork point, changes only on each side ✓ Objects changed on both ✓ Diff: CLI, REST, MCP ✓ Promotion: a branch's recorded statements, reviewed, then applied to another branch in one transaction	✓ Refused on a conflict (both sides changed the same object), on a statement the target's gate blocks, and when there is nothing to promote ✓ Approving needs the right to manage the target, so an agent cannot approve its own request ✓ Promotion: CLI, REST, MCP and the web Requests page	Unit; v2 §8, §8b
✅	H19	**The catalog scan: what the event triggers did not record** ✓ A change made with the event triggers stopped is recorded as UNRECORDED ✓ Roles, memberships, settings, databases, event triggers: OBSERVED ✓ A setting or trigger change that switches the record off: kill-switch	✓ Recorded changes, renames, cascades and restarts add nothing ✓ Its memory is outside the database; every 5 minutes and `fox blackbox scan` ✓ Who and exactly when stay unknown	Unit; v2 §11

	ID	Feature · sub-features		Tested by
☑	H20	**Application identity: who a change was made for** (F1) ✓ An application registered against its account and an Ed25519 public key; tokens accepted from that account's own logins only ✓ `bb.act_for($1)` : signature, header, key, revocation, binding, audience, branch and times checked; one actor per transaction, guarded by a MAC ✓ The Blackbox records whom each change was for, beside who made it; reports never upgrade a state	✓ Copied into every running branch at once, a suspended one on wake, a new one before it is reached ✓ Rotation, revocation, and a key reported compromised from a moment ✓ The extension in every engine image (revision 2); a branch on an older image refuses with the way out ✓ CLI `fox app` , REST `/api/apps` , the console's Applications page; `for` in `fox blackbox` , `?with=for` and the Blackbox page	Unit; f1 suite; §11u; UI
◐	H21	**The audit log: pgAudit, with no values kept** (F6) ✓ Turned on and off by an admin (`fox audit-log setup` / `off`), saying what it costs; main and the standby restarted, branches on wake ✓ Default mode: schema and role changes by everyone, marked tables through `bb_audit` , no statement text, no parameters, errors as their code alone ✓ The log outside every database, in a dataset of its own with a quota; `fox check` warns before it is full ✓ `fox audit-log status` , `/api/audit-log` , the console's Audit log page ✓ Marking tables for writes and for reads, from the CLI, the API and the console; marks travel with branches ✓ The collector: raw lines into a hash-chained store with only a record's fields, a year's retention, raw files removed after two days; `fox audit-log show` and `verify` , the API and the console's records; each read in the security log ✓ Gaps and delivery, found from outside: free space, a collector stopped, a heartbeat that does not arrive, statement numbers that skip (a gap record in the chain), a mark taken off outside FoxByte; in `fox check` , `status` , the API and the console ✓ Forensic mode, with a reason, for every branch or one: statement text kept beside the chain (its hash in it), 30 days, purged for a person; retention settings; a reload, never a restart	✓ Forwarding to a SIEM from the store, over TLS: HTTPS as JSON lines (a bearer token in the header) or syslog (RFC 5424, framed as RFC 5425); each record with its place in the chain; statement text only when asked; delivery tracked, retried, and a finding when it fails ✓ Enterprise: the chain anchored outside the store (signed, every 10 minutes, copied off the machine), and checked against every anchor in every edition ✓ Enterprise: confirming personal data (point 4.6) marks its table's writes; the tables confirmed and not audited named in every edition, and marked together with `table --personal-data` ✓ SECURITY DEFINER functions naming marked tables named as not audited, in `fox check` , `status` and the console ✓ Enterprise: the evidence export for a period: coverage and limits as stated, each branch's marks and gaps, integrity and anchors, delivery, activity and every change; with the records and anchors to check it offline (step 6b) ✓ Reads audited for the logins that may unmask: `fox mask unmask` (masking step 3, O3) sets the account's role's `pgaudit.role` to `bb_audit_read` , so its reads of marked tables are logged, and only its	Unit; f6 suite; §11y–§11af; UI
◐	H22	**The hardening report** (point 3.2; FoxByte's own checks, not the CIS Benchmark) ✓ About 40 checks, basic and strict, across installation, files, logging, auditing, roles, connections, settings and backups: pass, fail, manual or n/a, each with what was found, why it matters and the fix ✓ Where FoxByte differs on purpose (TLS at the Gateway, statement text kept out of the log, local trust inside the container), the check says so rather than failing ✓ `fox hardening [--branch] [--level] [--failed] [--json]` , `GET /api/hardening` (admins), the console's Hardening page (filters, JSON, print to PDF); each report in the security log	✓ Enterprise: a fix for each failing check that is safe to make without more than a yes (settings by ALTER SYSTEM and a reload, the public schema's CREATE, BYPASSRLS, the data directory's mode), after a preview, checked again afterwards; the rest stay a person's, with the command (3.3) ✓ Enterprise: profiles *baseline* (basic checks) and *strict* (basic and strict), previewed with what each leaves to a person, applied fix by fix and recorded against the branch (3.4) □ A mapping to the CIS Benchmark, once licensed	Unit; §11ag–§11ah; UI

I Import & migration

13 of 13 done

	ID	Feature · sub-features		Tested by
☑	I1	**From Postgres** ✓ Schema and data via pg_dump	✓ Zero-table result reported as an error	Manual; no automated test yet
☑	I2	**From MySQL ≤5.7 / MariaDB** ✓ pgloader with type mapping (multi-arch image)		Manual; no automated test yet
☑	I3	**From MySQL 8.x** ✓ Native mysqldump path (caching_sha2 auth) ✓ Types, NULLs, unicode, PK	→ Other indexes, foreign keys, defaults	Manual; no automated test yet
☑	I4	**From MongoDB** ✓ Collection → typed table, exact name	✓ Nested objects / arrays kept as jsonb	VM §9, §10
☑	I5	**From a .sql dump** ✓ Runs the dump into a new branch ✓ Fails listing every failed statement (instance kept) ✓ Same for Postgres imports (I1)	✓ Automated test ✓ An ordinary account's import runs as that account, backslash commands off	Unit; VM §11, §11h
☑	I6	**From JSON / NDJSON** ✓ Columns inferred from top-level keys	✓ An invalid document fails the import, naming its element or line	Unit; VM §10, §11
☑	I7	**From CSV** ✓ Table from the header, exact column names	! Typed columns (all text today)	Manual
☑	I8	**Continuous replication & cutover** ✓ Initial copy + streaming (logical replication) ✓ Cutover keeps data (CLI) ✓ Status and cutover from REST and the Import page	✓ A replicating branch is never auto-suspended ✓ Automated test (VM §5b) and driven in the browser	Unit; VM §5b

☑	I9	**Guard handling during import** ✓ Guardrail paused, recording stays on ✓ Re-armed even after a crash ✓ The target keeps the grants `db_client` needs: preparing it drops and recreates `public`, which had left imported data readable by the import's own superuser and by no gateway client at all	✓ The guardrail's `dblink` survives the reset (and a pipeline's rename); older imported branches mend on their next start	Manual; integration_test.sh §5b, §9, §11i
☑	I10	**Live progress (CLI & SSE)** ✓ Live log and progress bar	! psql subprocess output not streamed	Manual
☑	I11	**Import REST endpoints** ✓ From a connection string or an uploaded file ✓ Target names validated (the import creates the branch through branch.Create)	✓ Private, local and container sources for admins only; metadata refused ✓ Held to the checked address: no DNS rebinding past the guard (unit)	Manual; unit; VM §1d
☑	I12	**Web Import page** ✓ Database and file panels with live console	✓ No longer claims a SQLite loader that doesn't exist	Manual
☑	I13	**Exact identifier fidelity** ✓ Case kept (`userId` , `leaseAiChats`)	! Field order of JSON sources not kept	Unit; VM §10

J ETL pipelines (experimental)

4 of 4 done

	ID	Feature · sub-features		Tested by
☑	J1	**Pipeline engine** ✓ Extract → raw schema → SQL models → tests, in a fresh branch ✓ `ref()` / `source()` templating → Dependency graph → Incremental models	→ Scheduling ✓ A run replaces only a branch the pipeline made ✓ API sources are connection strings, never server paths ✓ An ordinary account's run is its own: extract, models and tests (unit)	Unit; VM §9
☑	J2	**Data-quality tests** ✓ not_null, unique, accepted_values, row_count_min, custom	✓ Failure → exit 1, data kept	Unit; VM §9
☑	J3	**Stored pipelines & run history** ✓ Saved per user, run over the API, history → Read a run's log	→ Mark runs whose process died	Manual
☑	J4	**Pipeline editor (web)** ✓ Create, edit, run, results, history	→ Confirm before delete	Manual

K APIs, SDKs & agents

6 of 6 done

	ID	Feature · sub-features		Tested by
☑	K1	**Control-plane REST API** ✓ Branches, query, Blackbox, policy, impact, import, pipelines, keys ✓ Versioned paths: `/api/v1/…` reaches the same routes, and `/api/…` keeps working	✓ Exact-origin CORS, branch-name check, body limits, timeouts ✓ Branches owned by their maker; others 404 (owner, admin, or main)	Unit; VM §1; v2 §0–§8; VM §1d
☑	K2	**OpenAPI spec** ✓ Served, Blackbox 2.0 endpoints documented ✓ Matches the code (import fields, status codes, auth routes, /blackbox aliases)	✓ Drift check in CI	Unit (drift check)
☑	K3	**Python SDK** ✓ Branches, query, Blackbox → Import, pipelines, keys, agents ✓ Contract tests against a live engine (nightly sdks suite)	→ Publish to PyPI ✓ Contract tests against a live engine	Manual; VM sdks
☑	K4	**TypeScript SDK** ✓ Same client for Node and browsers ✓ Contract tests against a live engine (nightly sdks suite)	→ Publish to npm ✓ Compiles, and contract tests against a live engine	Manual; VM sdks
☑	K5	**Agent Branch API** ✓ A branch per agent, own non-superuser role ✓ Provenance (task, parent, session) ✓ DSN reachable from the host (gateway-routed, branch-scoped key)	✓ Cap, listing and TTL count suspended branches ✓ Superuser switch only in `-tags insecure` builds ✓ Agents and their keys belong to the caller	Unit; VM §5; v2 §1c, §7
☑	K6	**MCP server** ✓ Branches, run_sql, Blackbox tools ✓ policy_check, execute_change, impact, diff ✓ Requires an API key and acts as that account; a scoped key is held to its branch	✓ Client config & 16-tool reference (docs/mcp.md) ✓ Same ownership rule as the REST API	Unit; v2 §1c–§8

L Web console

11 of 11 done

	ID	Feature · sub-features		Tested by

☑	L1	**Embedded single-origin UI** ✓ Served by the engine at :8080 ✓ Automated UI tests (Playwright, in the nightly ui suite)	✓ One branch-list loader for every selector: retries a dropped request, reports one it cannot load	Manual
☑	L2	**Login & signup** ✓ Log in / sign up tabs, OAuth buttons	✓ Covered by the console's Playwright tests	Manual; VM ui
☑	L3	**Dashboard** ✓ Live tiles ✓ Branch table: create, suspend, resume, delete, copy DSN	✓ Covered by the console's Playwright tests	Manual; VM ui
☑	L4	**SQL console & browser** ✓ Schema sidebar, rows, structure, indexes ✓ Values shown as Postgres writes them (uuid, interval, inet, bytea) ✓ SQL editor ✓ System tables hidden by default, with a show button ✓ Runs as the signed-in user; one-run override switch ✓ Per-rule override for policy blocks ! One statement per run (extended protocol) ✓ Runs a script, not just one statement: a result per statement, and one transaction unless it opens its own ✓ Copy a result, ticked rows or a cell as TSV, CSV, Markdown, JSON or INSERTs; spreadsheet-safe; `fox sql --format` writes the same (point 1.1) ✓ Export what is on screen as CSV, JSON or an Excel workbook; `fox sql --format xlsx` (point 1.2) ✓ Export a whole result, every row, streamed, read only, as the caller, recorded in the security log; `fox export` (point 1.3) ✓ Query history: your own SQL, branch, time, duration, rows and outcome, never a result; 90 days; can be off; deletable; `fox history` (points 8.1, 8.3) ✓ History panel: search, failed only, this branch; open, run again where it ran, copy, save; saved queries; `fox history --search|show|run|save|saved` (point 8.2) ✓ Query suggestions: autocomplete of tables, columns and words; "did you mean" with a one-click fix, also in `fox sql` ; snippets for a table (points 9.1–9.3) ✓ Object browser: schemas, tables (constraints, indexes, triggers), views, functions, procedures, sequences, types, extensions, each with its CREATE script; search; `fox schema` (point 12.1) ✓ Data grid: sort, filter, page; add, change and delete rows on a table with a primary key, saved together in one transaction as the user (point 12.2) ✓ Sessions and upkeep: the Sessions page and `fox sessions` list a branch's client sessions (login, state, what each waits on and who blocks it, its statement); a running query cancelled or a session ended, in the security log; the owner and admins see and stop everyone's, anyone else their own and their agent keys'; FoxByte's own never ended from here. A table's Upkeep tab and `fox maintain` : rows, dead row versions, size, last vacuum and analyse, and VACUUM, ANALYZE or both, the owner's or an admin's; VACUUM FULL not offered (point 12.5)	✓ Run, toggle and plan: a function or procedure run from a form in the object browser, one field per argument (NULL, or its default), the call shown as it is built and run as the person; a table's own triggers disabled (after asking) and enabled from its Triggers tab, FoxByte's own not offered; Explain in the Console, the plan as a table with each step's share and the most expensive marked, and with Analyse measured inside a transaction that is rolled back (point 12.4) ✓ Functions, procedures, trigger functions and triggers made, edited, renamed and dropped from dialogs in the Console: each shows the SQL it will run and runs it as the person, recorded in the Blackbox (a dropped procedure or trigger by the next catalog scan, as OBSERVED, as any such drop is); an edit keeps what it has no field for (SECURITY DEFINER, COST, SET clauses, a disabled trigger stays disabled); FoxByte's own are not offered; what a guardrail refuses can be put in the editor (point 12.3, first part) ✓ Row tracking: created_at, updated_at, last_modified_by (the login) kept by a protected trigger; opt-in per table or for new tables; `fox track` (points 5.1, 6.1) ✓ Row tracking's last_modified_for: whom each row was changed for, asserted by an application or claimed, never a value set by hand; added in place to a table tracked before it (point 5.2) ✓ Each result says which statement produced it; labelling is dropped rather than guessed when a split is unsafe ✓ Editor is read-only while a query runs, and ⌘/Ctrl+↵ no longer starts a second one ✓ Workbench layout: fills the window, schema rail folds away, results in tabs, either pane expands ✓ Line numbers, and a syntax error reports its line and column and marks the gutter ✓ Component tests for the console run on every pull request, without a stack ✓ Page heading kept for screen readers after the title block became a toolbar	Manual; API in v2 §6
☑	L5	**Blackbox page** ✓ Entries per branch with badges ✓ Table / risk / date filters, and "Clear filters" ✓ Verify button; a row expands to its statement and session ✓ Live: new entries arrive on their own (3s), marked briefly, pausable; never while older pages are being read	✓ The chain is visible: entry id, its row hash and the previous entry's, each copyable ✓ "Branch from before this change" on the row itself (`main` ; explained elsewhere) ✓ Covered by the console's Playwright tests	v2 §5b; browser; VM ui
☑	L6	**API keys page** ✓ Create, copy, list, revoke ✓ A new key is shown inside its connection string (E3's next step)	✓ "For an AI agent" mints an agent key; a Kind column ✓ Covered by the console's Playwright tests	Manual; browser review 21 Sep; VM ui
☑	L7	**Landing, Guide & Docs pages** ✓ Landing, developer guide, reference	✓ Refreshed against the code (https, sslmode, MCP, Blackbox, import routes)	Type-check + build
☑	L8	**Blackbox integrity page** ✓ Verify, checkpoint, export, branch-before ✓ Used signed in; every result matched the CLI	✓ New entries arrive on their own (3s), marked briefly, pausable	Type-check + build; browser review 16 Sep
☑	L9	**Policies page** ✓ Rules, warn/block, custom rules, preview, evaluations ✓ Used signed in; matched the CLI, wrote only to the chosen branch	✓ Non-admin change refused with a readable message ✓ Who can override: list, grant, revoke	Type-check + build; API in v2 §6; browser review 16 Sep
☑	L10	**App sidebar layout** ✓ Every link opens its page, highlighted, with breadcrumb ✓ Collapse persists across a reload; theme toggle persists	✓ No console errors on any page	Type-check + build; browser review 16 Sep; the under-860px drawer needs a manual look

	ID	Feature · sub-features		Tested by
☑	L11	**Design system & FoxByte theme** ✓ Tokens for colour, type, space, radius, elevation and motion; components use no raw values ✓ Ember palette in both themes; filled controls legible (`--accent-ink`)	✓ Logo artwork as the mark, wordmark from brand.json accenting the logo's letter, favicon and touch icon ✓ Console, Blackbox, Policies and API keys re-laid out; one table style ✓ Every page reviewed in both themes; console checked at 375px	Type-check + build; browser review 21 Sep; not yet reviewed against real data

M Build, CI & releases

8 of 9 done

Policy authoring, impact, promotion, export and pipelines gated; reading each stays free
All five moved: `enterprise/{pipeline,impact,promote,anchor,policy}` — ~950 lines, and zero enterprise symbols in the Standard binary
SQL schema split: `ledger_ext` , `provenance` and `datachanges` stay free; `checkpoints` , `policy` and `impact` moved to `enterprise/schema` and are installed per feature
Realtime foundation: opt-in, container settings (byte-identical when off), slot naming and sweeping, the suspend probe, preflight refusals
Realtime decoder: wire format, fan-out, pgoutput decoding — all in `enterprise/realtime`
SSE route on the outer mux, branch-scoped key (`UserForBranchStream`), decoder lifecycle, `realtime enable|disable|tables` , console page
Realtime integration suite (`make integration-realtime` , in the nightly matrix): refusals, the wire format against a real WAL, scoped keys, slot sweeping, teardown, replay via `?since=` , and the WAL budget invalidating a slot

	ID	Feature · sub-features		Tested by
☑	M1	**Makefile targets** ✓ build, vet, test, integration suites ✓ Suites run only in a throwaway test VM (guarded)	✓ release, web, WSL builds, feature-doc (both PDFs) ✓ `integration-sdks` , `integration-ui`	CI
☑	M2	**Linux CI** ✓ Build, vet, test, gofmt ✓ Release script tests, checklist sync check ✓ VM integration suites nightly in CI (one runner per suite) ✓ Release tags gated on them (release.yml nightly-gate)	✓ Secret scanning, govulncheck, npm audit, UI type-check ✓ No suite passes having tested nothing, or fails on another suite's leftovers	CI
☑	M3	**Windows CI** ✓ Build, vet, test, Pester, installer encoding	✓ Guard tests read backslash paths, and generated files are pinned to LF (21 Sep)	CI; no end-to-end WSL install test yet
☑	M4	**Releases & GHCR image** ✓ Binaries for 5 platforms + fox-verify ✓ SHA256SUMS merged safely between jobs ✓ Fails unless `fox update` 's files are published ✓ Multi-arch Postgres image on GHCR	✓ Actions pinned by SHA; signed SHA256SUMS; build attestations; SBOM ✓ Run for a real release on GitHub, and verified afterwards ✓ go.mod names the toolchain releases use (go1.26.9)	Script tests in CI
☑	M5	**WSL distro & ZFS builds** ✓ Prebuilt distro image ✓ Downloaded and verified by the installer (seen on Windows) ✓ Preloaded image tagged as the engine runs it	✓ Imported on a fresh Windows install; preloaded images loaded ✓ Published zstd-compressed, with the gzip size measured beside it in every build log	Actions artifacts; Pester; Windows run 16 Sep; the zstd unpack is unit-tested only, not run on a real Windows install
☑	M6	**Postgres image** ✓ PostgreSQL 18 + wal-g for a fresh install; one image per supported major (16, 18) ✓ The image follows the data: an install on 16 keeps running 16 after an update ✓ A branch on the wrong major is refused before `docker run` , with the way out ✓ The major lives in one place, and a test holds the Dockerfile, release matrix and Windows preload to it	✓ All three suites pass on PostgreSQL 18.6: 108 + 277 + 51 checks (21 Sep) ✓ A command that moves an install from 16 to 18 (D6), and a portable backup to take first (D5) ✓ pgvector, version-pinned from PGDG for both supported majors — available, and enabled per database with `CREATE EXTENSION vector` ✓ Base image pinned by digest; wal-g checked by SHA-256	Unit; all three suites on 18
☑	M7	**Product name in one file** ✓ brand.json → Go, shell, web and installer blocks, regenerated by `make brand` ✓ Stored names brand-free and frozen: a rename never migrates an install	✓ Retired env prefixes, state directory, VM names and installs still handled ✓ Guards: retired names anywhere, product names in stored SQL, durable names spelt twice ! The sweep is text substitution: review its diff (it over-reached twice)	Unit (brand, gen, oldnames), in CI via go test
☑	M8	**Two editions** ✓ One tree, two binaries: `fox` (Standard, AGPL) and `fox-enterprise` ✓ `enterprise/` has its own commercial LICENSE and `LicenseRef-` headers ✓ The Standard binary contains no paid code — checked with `go tool nm` , not just the source ✓ An Enterprise build with no licence behaves exactly like Standard ✓ Installer `--edition` / `FOX_EDITION` ; launcher and VM engine always match	✓ `fox update` stays inside its edition; the release publishes and checks both sets ✓ `fox upgrade --edition enterprise|standard` switches an install at the same version, licence accepted first ✓ Install Enterprise from the console (admins); `fox` on macOS/Windows catches up on its next command (not on `version` or `help`) ! Source-available deters, it does not prevent: the check can be removed in breach ! A dependency only the paid code uses still appears in the AGPL module's `go.mod`	Unit (edition, boundary), scripts/test_editions.sh, CI builds both

M9	**Licensing**		internal/license

- ✓ Activating from the console: `POST /api/license` and an Account → Licence page — admin only, a forgery refused outright, honoured by this process at once, with the restart and the VM-copy caveats stated rather than hidden
- ✓ **Upgrading from the navbar**: a badge on every page says what the engine is honouring — Standard (also on an Enterprise build with no licence), Enterprise, Enterprise in amber when it wants attention, Licence lapsed in red — and an admin sees **Upgrade** beside it while the paid features are locked. The modal takes the licence as an uploaded file or pasted text, shows a spinner while it activates, and ends in one of three outcomes: a green tick and “Enterprise activated”, “activated, with a warning”, or “installed, but nothing is unlocked” — never a tick on an expired licence. On a Standard build it shows how to install the Enterprise build instead, since no licence can unlock code that is not there. Closing it unlocks every lock on the page without a reload. Walked through in a real browser, and a Playwright test does it with a real file upload
- ✓ **A licence is honoured without a restart**: every long-running engine process watches the licence file and re-reads it on change and every ten minutes (for expiry and grace). Measured: an activation in one control plane is honoured by the separate Gateway, Agent API and control plane within four seconds, with the same process ids. The entitlement is atomic now, closing a data race
- ✓ **One licence, two copies, kept in step** (macOS/Windows): a licence activated in the console is kept on the host at the next `fox start` or `license show`, instead of being lost to a VM rebuild or overwritten by the host's older copy; a removal made with the VM stopped is finished at the next sync instead of never reaching the engine. The most recent activation wins; a copy that does not verify never does
- ✓ `make integration-ui` passed its email and password to Playwright through a command with a comment in the middle of its line continuations, so they never arrived; it worked because the defaults matched. Fixed
- ✓ Ed25519 over a canonical payload, with a key id so the key can rotate
- ✓ `cmd/licensesign`: generate, issue, verify; private key never in CI
- ✓ Soft machine lock, grace before refusal, and a lapse that never stops a database
- ✓ Host fingerprint: hashed, read on the host not in the VM, failure means unknown
- ✓ `~/.fox/license.json` (0600, atomic) and `fox license activate|show|rebind|remove`
- ✓ The engine inside the VM is given a copy (macOS/Windows); no-op on Linux
- ✓ Security-log records: `license.{activated,rebound,lapsed,removed}`, written by the engine at start-up
- ✓ `GET /api/license` (state for everyone, customer and binding for admins)
- ✓ The console's notice: silent when active, silent in Standard, dismissible only while it still unlocks
- ✓ Anchors gated: writing is Enterprise, checking (`integrity`, `fox-verify`, `anchor-key`) and the security log's anchors stay free

N Realtime change feed

1 of 1 done

ID	Item · scope	Tested by

✓	N1	**Realtime change feed**	enterprise/realtime, scripts/integration_realtime.sh

- ✓ Readiness as a verdict, not a refusal: `realtime doctor` classifies every table, `realtime prepare` makes the changes and prints them first
- ✓ The cost ladder — primary key, then a unique index as the identity (same price, no schema change), then inserts-only, and `REPLICA IDENTITY FULL` last; `--all` never applies a costly fix
- ✓ Only application tables stream: not `bb`, not Postgres's, not an extension's — and the application's own schemas now do
- ✓ Opt-in: `fox realtime setup --yes` says what it will restart; with the feed off, a primary's arguments are byte-identical to before and a branch gets none
- ✓ Durable replay through `?since=<commit_lsn>`, bounded by `max_slot_wal_keep_size`; past it the subscriber is told to `resync` rather than left waiting
- ✓ One decoder per branch in the control plane, started on the first subscriber and stopped after the last; it wakes a suspended branch
- ✓ **A `?since=` is judged before the stream opens** (21 Oct 2026, found reviewing the realtime plan): it went to the decoder unchecked, and was silent both ways — a position the feed had read past attached the subscriber to the live stream with a hole in its copy and no event to say so, and a position *ahead* of the branch was reported to Postgres as flushed, so one request moved the slot past the end of the WAL and the branch's feed then delivered nothing to anybody, across restarts, with `fox realtime slots` still calling the slot idle. Four answers now: replay, join live, 400 when ahead of the branch's log, `resync` / `cursor_expired` when it cannot be replayed; a running fan-out keeps the floor it can honour, and the decoder checks the log end before acknowledging anything
- ✓ **An overflow inside one transaction says to refetch, not to resume**: pgoutput proto 1 sends a transaction whole, so one larger than the buffer cannot be delivered at all, and the old advice replayed it for ever (measured: 31 of 100,000 changes, then 2) or skipped the rest of it silently. `resync` / `tx_too_large` names the size and the buffer; `FOX_REALTIME_BUFFER` raises it. The overflowed subscriber is also held attached until the decoder has acknowledged that transaction, so the feed moves past it instead of the next subscriber replaying it and overflowing in the same place. Falling behind across transactions is unchanged
- ✓ Refusals that hold: row-level security (no override), no primary key (and `UPDATE` still works afterwards), anything outside `public`, the Blackbox's own tables, anything `db_client` cannot read
- ✓ Wire format: values as text, identity always, `old` only when sent, `changed` only under FULL, unchanged-TOAST named rather than nulled
- ✓ Both answers to "what was the row that is gone", in one capture against a live WAL: a delete under the default identity carries the key alone, and under FULL the whole old row — where every column is part of the identity too
- ✓ SSE on the outer mux so a branch-scoped key reaches its own branch and nothing else; somebody else's branch is 404, not 403
- ✓ Slots named, listed, accounted for and swept only once Postgres has marked them lost; `fox check` warns on one nothing is draining
- ✓ The reaper leaves a subscribed branch alone (asked of Postgres, not of memory), and a resumed branch gets a fresh idle window
- ✓ A failover keeps the feed: the standby carries the same arguments, `setup` restarts one that predates it, and the feed follows `main` onto the promoted standby
- ✓ A subscriber's **position** survives the failover too (HA plan phase 4): failover-aware slots synchronised to the standby through its own physical slot, so a change written before the promotion is delivered after it instead of costing a refetch — gated on the PostgreSQL major, since the settings are fatal on 16
- ✓ …and the limit is stated rather than implied: PostgreSQL copies a slot to the standby only while it is being consumed, so a continuously-read subscriber keeps its place through a promotion and one that read once and left is told to resync, as it was before
- ✓ Neighbours unaffected: a continuous import runs under logical decoding and is itself decodable; backups, verify, failover and failback all pass
- ✓ `setup`, `teardown`, `status` and `slots` stay in the free build, so a downgrade can still undo it
- ! Console and REST only this release; the published SDKs get the feed once the wire format has survived real use
- ! No per-subscriber row filters, which is why an RLS table is refused rather than filtered
- ✓ Published clients subscribe: `clients/typescript` and `clients/python`, dependency-free, with whole-transaction delivery, resume only at a commit boundary, reconnection, and an `apply` that leaves an unchanged TOASTed column alone
- ✓ 41 client unit tests (18 TypeScript via `node --test`, 23 Python via `unittest`) plus contract tests driving both against a live engine with a real `rtk_` key
- ✓ Transaction framing (`?transactions=1`): `begin` / `commit` around each transaction, every change carrying its `xid` and the transaction's `commit_lsn` as the resume boundary, with the record's own position as `lsn`
- ✓ Off by default and dropped per subscriber, so an unframed client is unaffected; an empty transaction produces no frame; a value other than 1 or 0 is refused rather than assumed
- ✓ What staying warm costs: `realtime activity`, an API route and a console panel — warm-since from the container, work from `pg_stat_database` sampled on a timer, and the bookmarks' held WAL against the budget
- ✓ Transactions rather than queries, because that is what Postgres counts without `pg_stat_statements`; and five cases where subtracting two readings would lie are each answered with a note instead of a figure
- ✓ A cap of 8 subscribers per branch (`FOX_REALTIME_MAX_SUBSCRIBERS`), refused with the count, so a client reconnecting on a bug cannot add streams until something runs out
- ✓ Console: Tables, Connecting an application, and Feed — verdicts become buttons, and the one fix with an ongoing cost asks first with the measured figure in the dialog
- ✓ Console routes on the inner `/api/` mux (`enable`, `disable`, `prepare`, realtime keys), all `Manage` by prefix; reading verdicts reuses the front door so the console cannot drift from what an application sees
- ✓ Covered in a browser: `web/tests/realtime.spec.ts` classifies a real branch, applies a free fix, declines a costly one, publishes and unpublishes a table, and mints then revokes a key
- ✓ Front door: `/realtime/v1/branches/{name}/{hello,tables,stream}` on the console's port, and a `fox-realtime://` connection string that keeps the key in userinfo, never in a URL
- ✓ A `realtime` key kind (`rtk_`, `fox realtime key create|ls|revoke`): streams one branch, refused at the control plane and **refused at the Gateway by kind** — asserted against a real psql connection
- ✓ Only an application's tables over HTTP: one shared `realtime.Application()` withholds `bb`, Postgres and extension-owned tables and reports how many — the first version served the Blackbox's shape to a stream-only key
- ✓ Additive: the original `/api/branches/{name}/realtime` route is unchanged, and every key issued before kinds existed keeps the empty kind and its old behaviour (migration covered from a pre-`kind` store)
- ! A delete under the default replica identity carries the primary key and nothing else

O Personal data & compliance

3 of 3 done

ID	Item · scope	Tested by

01 **Personal-data discovery: scan, review, keeping it current** (points 4.1, 4.2, 4.6, 4.7)

enterprise/pii unit; VM §11r; personaldata.spec.ts

- ✓ Recognisers for Aadhaar (Verhoeff), PAN (holder type), GSTIN (check character), mobile, card (Luhn), email, UPI, IFSC, passport, voter ID, vehicle registration, date of birth, IP address; a date of birth under 18 is a child's data
- ✓ By name and by value, with a confidence: a checked value alone, a format with most of the sample or the name, a weak shape only with the name; chance checksum passes are not findings
- ✓ Up to 1,000 rows a table (TABLESAMPLE on large ones), read-only, 30 s per table, as the superuser so row-level security hides nothing from it
- ✓ No value stored, logged or shown; findings in the branch, not granted to clients, Manage to read or scan; each scan in the security log
- ✓ A finding found again keeps its state; one no longer seen is kept and marked
- ✓ JSON documents walked path by path, each path classified like a column; arrays as `[*]`, numbers kept as digits; bounded at 256 KB a document, 32 levels and 500 paths a column, each bound reported when reached
- ✓ Review: confirm, reject or reopen each finding with a note; the latest on the finding, every decision in an append-only record; kept across scans, carried by branches; in the security log
- ✓ Masked samples to decide by, read as the reviewer's own role (row-level security and grants apply), masked in the engine, never stored; JSON paths followed by Postgres
- ✓ Keeping it current: an inventory of columns and JSON paths; the first scan is the baseline, and what appears after it is unclassified until its findings are decided or it is accepted; kept append-only, in the security log
- ✓ A quick scan of what is new (`--new`): only tables with an unseen column, plus JSON columns, and says what it left; "not seen" judged against the latest full scan
- ✓ `fox pii scan|findings|show|samples|mark|unclassified|accept`, seven API routes, and the console's Personal data page with its review queue and Unclassified tab
- ! A sample, not every row; free text (4.3) is a later step; marks do not yet create pgAudit grants (F6)

02 **Access review report** (point 4.8)

enterprise/accessreview unit; VM §11s; accessreview.spec.ts

- ✓ Accounts: install admin or not, what each reaches, owned branches, guardrail overrides, keys, last active (sign-in, key use, query), dormant after a window
- ✓ Keys: kind (account, agent, realtime) and what that means, reach, age, last use, stale and why; never a secret
- ✓ Branches: owner, and who may override their guardrails (running branches; a suspended one not woken, and said so)
- ✓ `fox access review [--dormant] [--json] [--csv accounts|keys|branches|applications]`, `GET /api/access-review` (admins only), and the console's Access review page with CSV downloads and print for PDF
- ✓ Each report in the security log; what it cannot claim stated in it
- ✓ Applications: each registration, its account, its keys with age, state and what each asserted by branch, and findings to question; `--csv applications` (F1)
- ✓ Sees unmasked: whether each account is an admin, not masked, or masked, on the branches it owns too (step 4), and where it holds unmask, its reads audited (masking step 3)
- ! Campaigns are 4.9 (Gate 2)

03 **Masking personal data** (point 4.6 and F5, Enterprise; `docs/FOX_F5_Masking_Design.md`)

- ✓ Gate 0 prototype (`make integration-f5`, 57/57 on PostgreSQL 18 and 16): the keyed hash in `bb_identity` 0.2, its key a file the server reads; a masked branch on a ZFS clone left with no real value in its files; the costs measured
- ✓ Step 1: `fox mask setup|rule|account`, four API routes and the console's Masking page; rules (partial, keyed hash, fake, empty) only on confirmed findings, Aadhaar never hashed or faked; masks kept in shadow tables by statement triggers; masked accounts read masks and ordinary tables, never the real marked table, the hash or the key, and cannot write to marked tables; admins see real values (a branch's owner did too, until step 4); the cost said before a rule is set
- ✓ Step 2: `fox mask key [status|create|rotate|drop|use]`, two API routes and a Keys panel on the Masking page; key domains whose masks hash apart; a table's masks put under one; a key rotated on every running branch while masked readers read the old masks until each table is done and writes pause only for a moment, then the old key deleted; a rotation cut short carries on when run again; a branch stopped through a rotation named, and its masks made again by setup; a domain in use, or perhaps in use on a branch stopped since it was made, is not deleted
- ✓ Step 3: `fox mask unmask <email> --branch b` gives a masked account real values on one branch, every read it makes there of a marked table logged by the audit log through `bb_audit_read` (its role's `pgaudit.role`, a superuser's setting it cannot undo), which closes 3.1; the lookup, `bb_mask.find(table, column, value)`, finds a person by an exact value and returns the rows masked, 30 a minute for each account, each call recorded without the value (`fox mask lookups`); the access review's Sees unmasked column; a deleted branch's unmask goes with it; two API routes and the Masking page's Unmask and Lookups tables
- ✓ Step 4: every access path the design names tested as a masked account and as its agent key (integration §11al, masking.spec.ts). It found eight ways to real values, now closed: a branch it owns, made of main or back in time, reads masks (owning one no longer unmasks); its agent keys log in as a masked agent role, which cannot reach `db_client` by inheriting or `SET ROLE`, and MCP's `run_sql` and `execute_change` never fall back to a shared role; agent branches are refused to it (masked branches since step 5); realtime is refused to it where there are masks; the Gateway refuses it rather than log it in as the shared role; the Blackbox's entries reach it without their statements' text, and its export is refused; a guard in the database (`bb.mask_close_paths`, an event trigger) takes masked accounts' rights on a view or materialized view reading a masked column, on a marked table granted back, and `INSERT` / `UPDATE` on a table with a foreign key into a masked column, and gives them back when the rule goes, refusing a `GRANT` to `PUBLIC`; `fox mask` and the Masking page list what it closed and the definer functions that can return real values. Kept, and now tested: COPY, `RETURNING`, `ON CONFLICT`, `MERGE`, errors, `pg_stats`, `EXPLAIN`, the console's query, grid, object browser and exports
- ✓ Step 5: `fox branch create <name> --masked [--from b]`, the Dashboard's *masked* option and `POST /api/branches` with `masked` (made in the background, 202), `fox mask branches`, `GET /api/masking/branches` and the Masking page's Masked branches table. The copy is recorded before it exists, so the Gateway, the console, the API, MCP and the agents refuse it (423, 57P03) until it is masked and verified, and it never mounts the key; every marked column is overwritten with the mask a masked reader sees, checked row by row; the masks' machinery is removed; its Blackbox starts afresh, numbering on from the source's last entry, so a change request from it carries only what was done on it; `VACUUM FULL` of each masked table and the statistics; the source's replication slots dropped and the WAL from before removed. Verified: no WAL segment from before, nothing of the source's Blackbox, and the scan finding no masked category, nor anything confirmed, where no rule masked it (a finding rejected on the source is ignored). A failure, or a run stopped part way, leaves it closed, saying why; running it again starts over. On an open masked branch nobody is masked; a copy of one is one at once; reset makes it again; a masked account's agent branch is made as one (replacing step 4's refusal)
- ! One-column primary keys only; a JSON path into an array, or a quoted key, is refused for now; masking stays on once set up; a masked account has no DDL rights; a domain's key is the install's, shared by every branch, so a branch stopped through a rotation keeps its old masks until set up again; an unmask holder's reads are logged only while the audit log is on; the lookup is by a column with a rule of its own, not a JSON path; a definer function that can leak is listed, never changed; `CREATE TABLE AS` copies real values into an ordinary table, which the scanner finds on its next run; the change requests list shows each request's SQL to anyone on its branches; a masked branch's blocks shared with its source's snapshot, and the source's own WAL and backups, hold real values (they are the source's); on a masked branch a foreign key from a column without a rule into a hashed column no longer matches (masking skips the check)

enterprise/masking unit; internal/branch mask_test.go, maskkey_test.go, maskpaths_test.go, maskedbranch_test.go; VM §11ai, §11aj, §11ak, §11al, §11am; masking.spec.ts

Z Not started (roadmap)

7 items

Planned or discussed, nothing built yet. When work starts, move the row into its area with the next id there.

	ID	Item · scope	Related
☐	–	**Multi-tenant projects** ☐ Projects / organisations, and roles within them ✓ Per-branch permissions: owner or admin, on every branch-scoped route (G2 of the audit)	G1, H6
☐	–	**Automatic HA failover** ☐ Health detection, fencing ☐ Multi-host	D4
☐	–	**Pipelines: scheduling, incremental / CDC extract, DAG view**	J1
☐	–	**More import sources: Cassandra, DynamoDB**	I1–I8
☑	–	**Release signatures** — done 22 Sep 2026	A9, M4
☐	–	**Hosted cloud tier**	—
☐	–	**Networked storage disaggregation** (README roadmap; see the storage engine note on page 1)	C1, D1

Out of scope by decision

A custom storage engine (PostgreSQL's own engine is used, for native transaction speed), sharding, Raft / cross-node transactions, multi-region, HTAP.